TECHCATALYST DE 2026

Cloud fundamentals

The Data Engineer's view.

Service models, GCP and AWS side by side, identity & access, and how not to blow the budget.

Week 1
Day 2

Why cloud

Service models & responsibility

GCP & AWS for DE

Identity & pipeline auth

Networking, billing & cost

DAY 2

Today's agenda

Time	Block
8:00 – 9:00	Recap and stand-up: yesterday's readouts revisited
9:00 – 10:30	Why cloud, service models, shared responsibility, regions, project hierarchy
10:30 – 12:00	GCP and AWS for DE: capabilities, database landscape, compute, networking and egress
1:00 – 2:00	Identity and access: IAM and pipeline authentication
2:00 – 2:45	Billing and cost control; instructor console orientation
2:45 – 3:45	Activity 1 (BigQuery Sandbox) and Activity 3 (Pricing Calculator)
3:45 – 5:00	Activity 2: cloud architecture scenarios and readout

Why the cloud won

OpEx

Pay for what you use, with no upfront hardware capital

Elastic

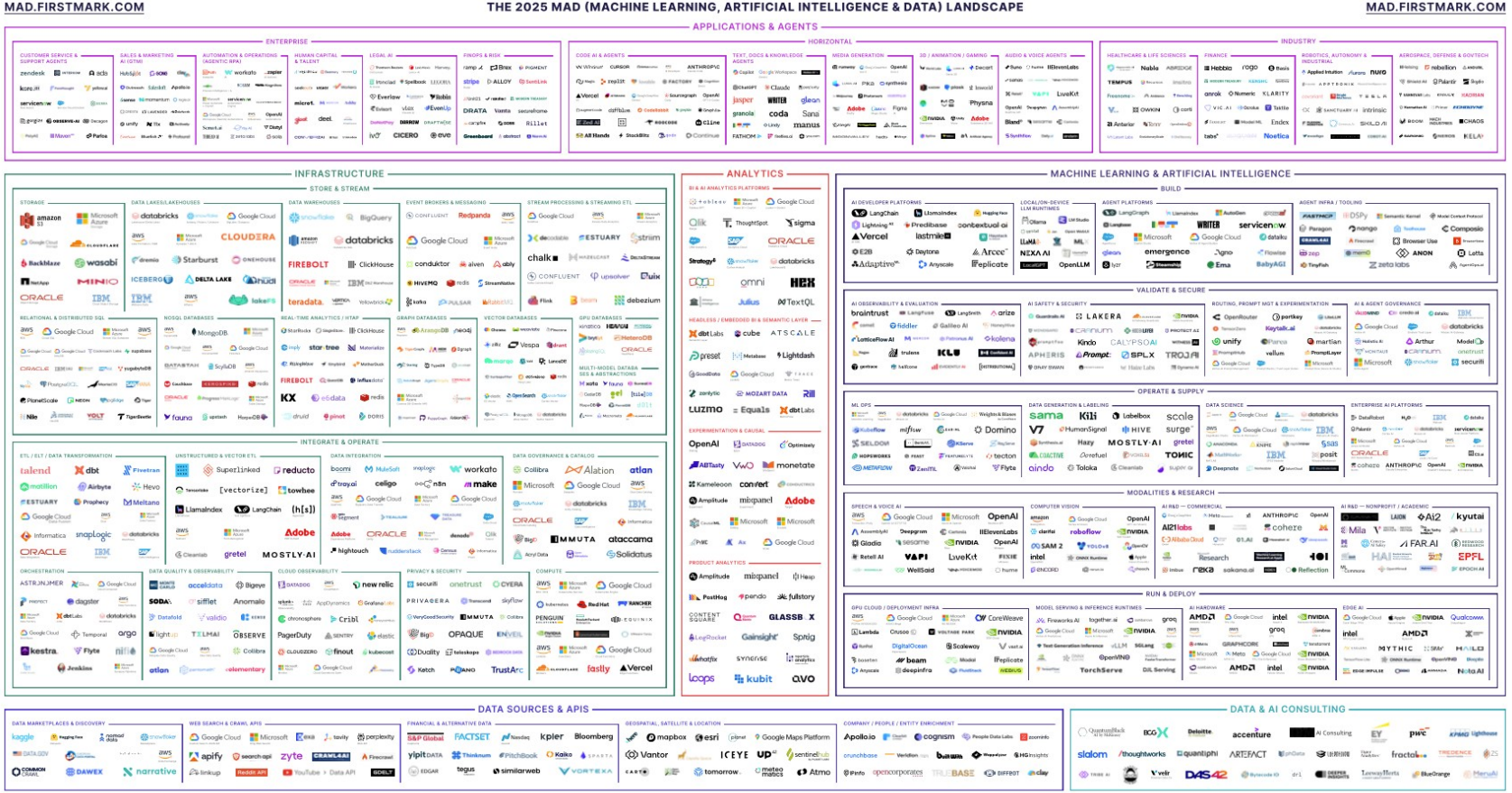
Scale from 1 to 1,000 machines for an hour, then back

Managed

Databases, queues, and Spark clusters as a service

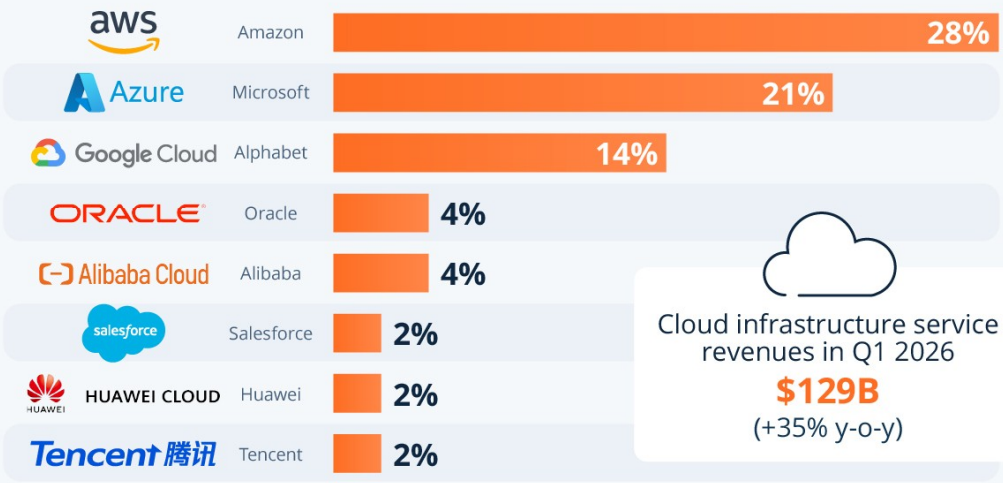
- Before cloud: order servers, wait weeks, over-provision for peak load, maintain everything yourself.
- With cloud: provision in minutes, experiment cheaply, let the provider run the undifferentiated heavy lifting.
- For DEs this means: more time on pipelines and data, less on hardware and patching.

DATA / DE LANDSCAPE



Big Three Hold Dominant Lead in Accelerating Cloud Market

Worldwide market share of leading cloud infrastructure service providers in Q1 2026*



* Includes platform as a service (PaaS) and infrastructure as a service (IaaS) as well as hosted private cloud services

Source: Synergy Research Group

statista

IaaS, PaaS, SaaS: who manages what?

Layer	On-prem	IaaS	PaaS	SaaS
Application & data	**You**	**You**	**You**	Provider
Runtime & middleware	**You**	**You**	Provider	Provider
OS & virtualization	**You**	Provider	Provider	Provider
Servers, storage, network	**You**	Provider	Provider	Provider

DE examples. IaaS: a VM you install Spark on (Compute Engine, EC2). PaaS: BigQuery, Dataflow, Snowflake. SaaS: Tableau Cloud, ThoughtSpot.

The shared responsibility model

Provider secures the cloud

Physical data centers and hardware

Network infrastructure

Hypervisor and host OS

Service availability (SLAs)

You secure what's in the cloud

Who can access your data (IAM)

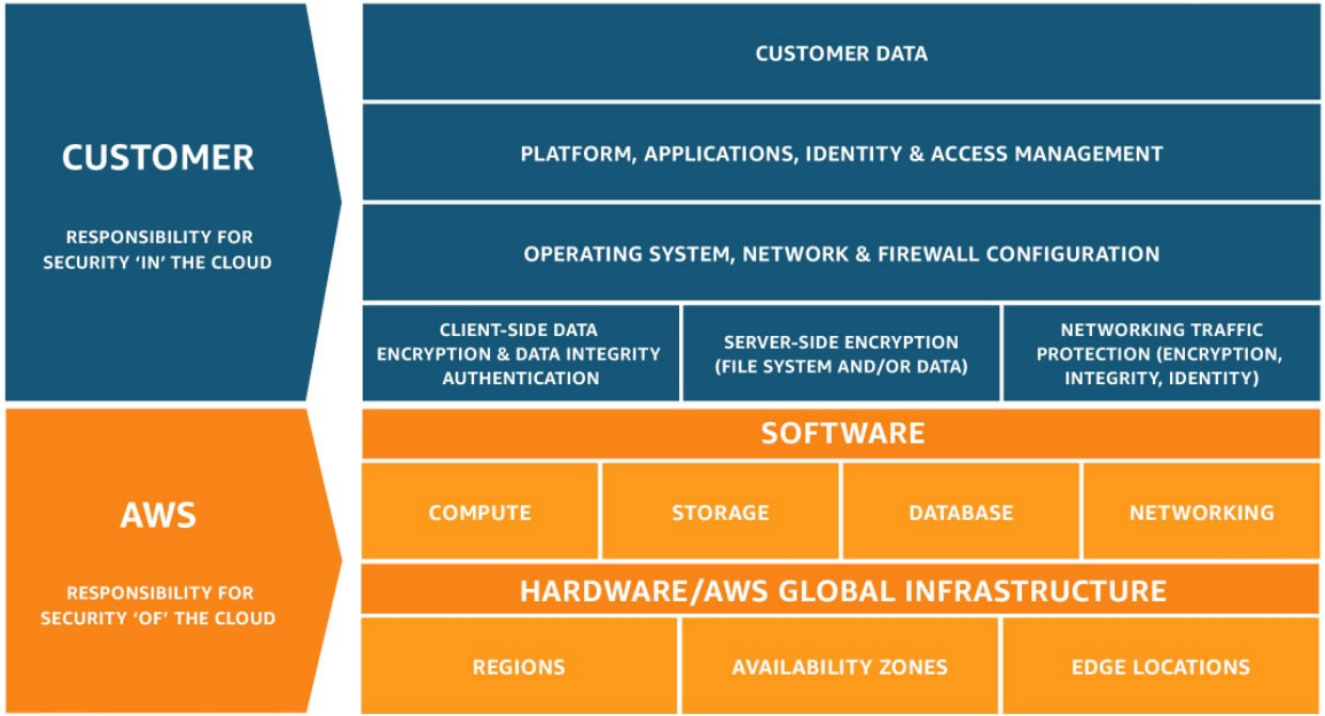
Bucket and dataset permissions

Encryption choices and key management

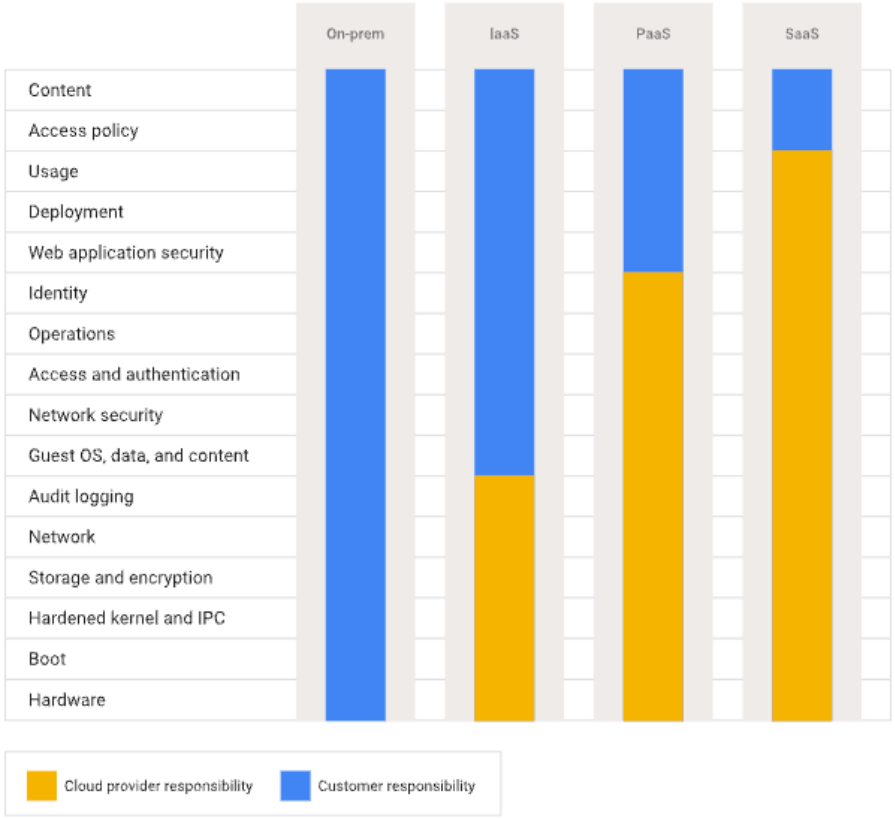
PII handling and compliance

Most cloud breaches come from misconfiguration, like a public bucket or an over-permissive role, not from provider failures.

The shared responsibility model (AWS)

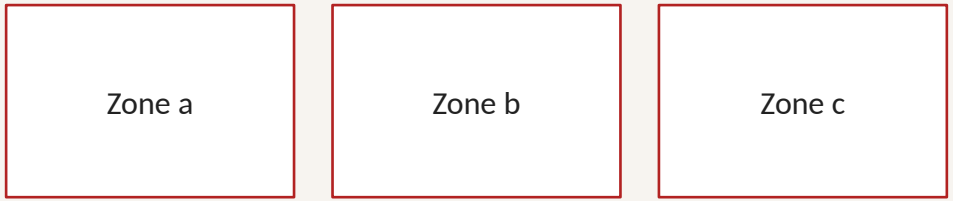


The shared responsibility model (GCP)



Regions, zones, and latency

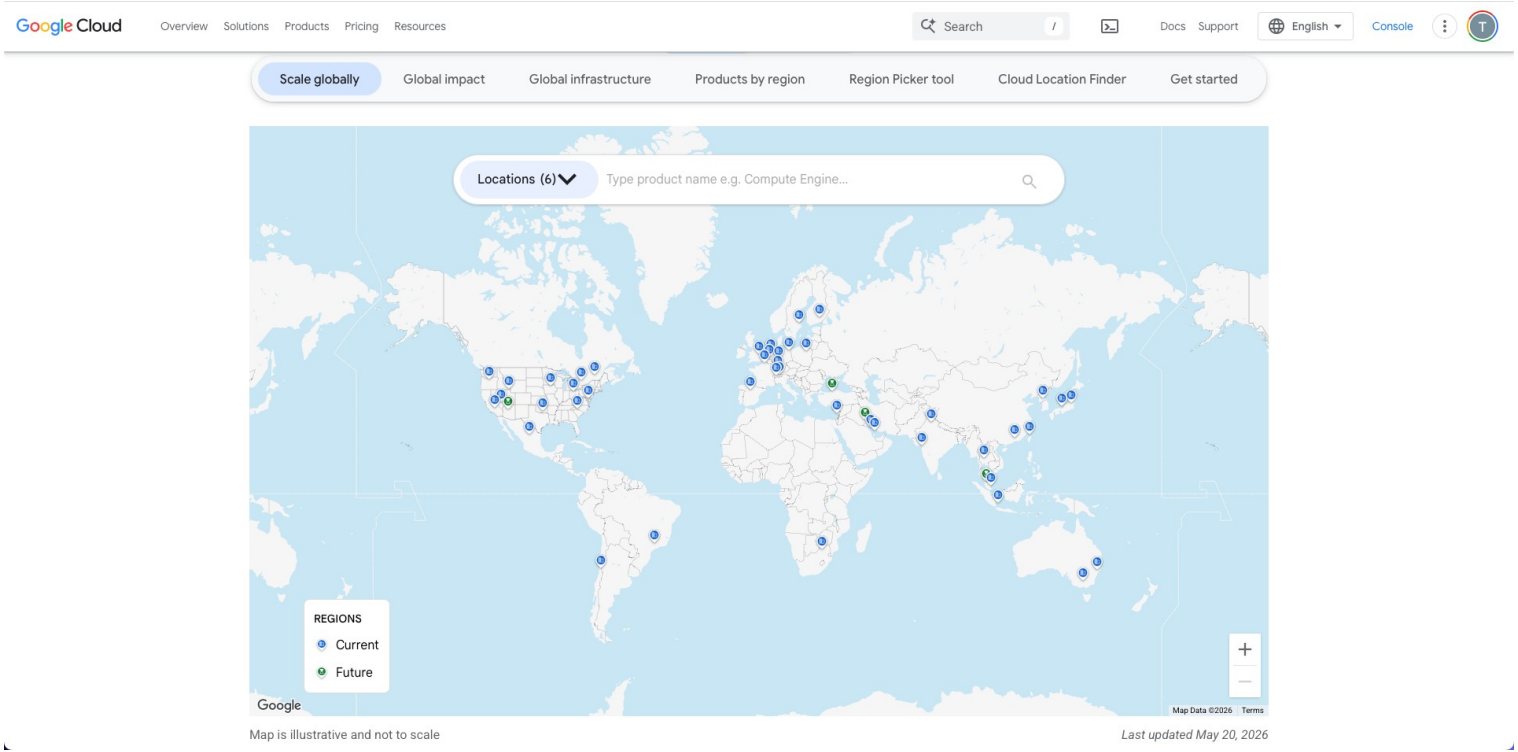
Region: us-east1 (South Carolina)



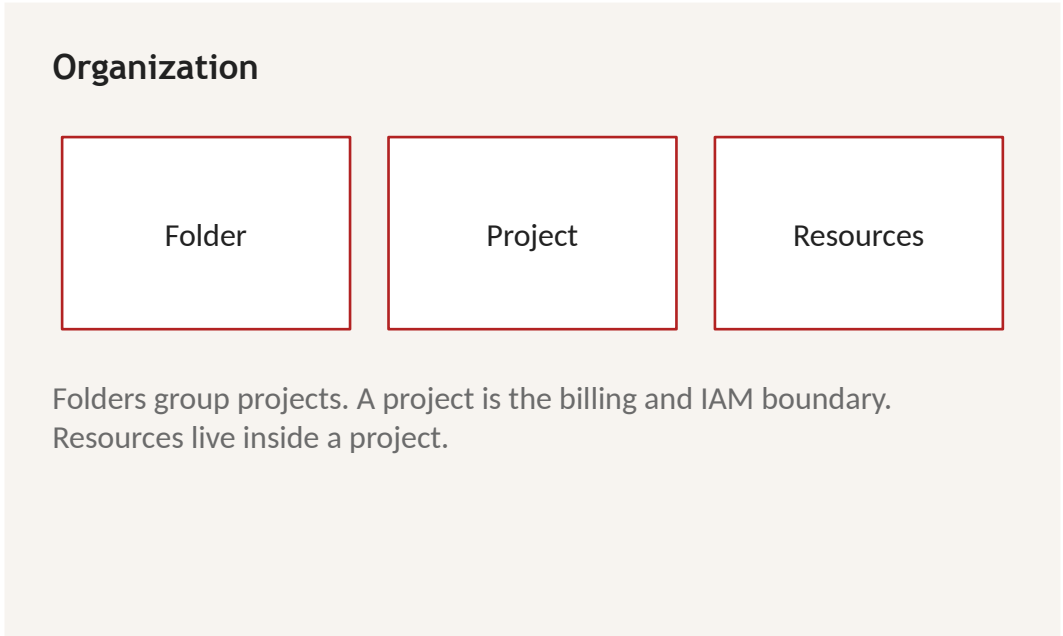
Zones = isolated data centers within a region. Multi-zone = high availability.

- Keep compute next to data; cross-region reads are slow and cost egress.
- Data residency: regulated data may have to stay in specific regions.
- Your labs: everything in one region to keep it simple and cheap.

Regions, zones, and latency



Projects are your blast radius



- Each project gets its own billing, IAM, and quotas, so a mistake stays contained.
- Use one project per team or environment to keep cost and access easy to reason about.
- GCP nests organization, folder, project, and resource. AWS uses accounts the same way.

Resource hierarchy at a glance

Layer	AWS	Google Cloud	Azure
Top-level org	AWS Organization	Organization	Entra ID Tenant
Grouping	Organizational Unit (OU)	Folder	Management Group
Core unit	**AWS Account**	**Project**	**Subscription + Resource Group**
Leaf	Resource	Resource	Resource

Each major cloud provider has a distinct organizational model. The key differentiator is what acts as the primary isolation and billing boundary.

Design philosophies: Billing & Isolation

AWS: Account-Centric

Uses the **AWS Account** as its primary boundary for both billing and security isolation.

Resources live directly inside an account with no mandatory grouping construct below that level (no native Resource Group equivalent).

Governance is handled via Organizations & SCPs, relying heavily on **tagging** for organization.

GCP: Project-Centric

In GCP, everything lives inside a **Project** — billing, APIs, IAM bindings, and resource quotas are all scoped here.

Folders provide an optional logical grouping above projects, and permissions inherit downward.

A GCP Project is roughly equivalent to an Azure Subscription + Resource Group combined.

Azure: Sub + RG Centric

Features a two-layer core structure. A **Subscription** acts as the primary billing and access boundary.

Inside, every single resource *must* belong to a **Resource Group**.

Unlike AWS, deleting a Resource Group deletes all resources inside it, making it a strict, hard organizational container.

01

GCP & AWS for data engineering

Two clouds, one mental model.

Core DE capabilities across two clouds

Capability	Google Cloud	AWS
Object storage	Cloud Storage	Amazon S3
Warehouse / SQL	BigQuery	Redshift · Athena
Messaging / events	Pub/Sub	Kinesis · SNS / SQS
Batch / stream	Dataflow	Glue · Firehose
Managed Spark	Dataproc	EMR
Orchestration	Cloud Composer	MWAA · Step Functions
Catalog / governance	Knowledge Catalog	Glue Catalog · Lake Formation
AI / ML	Vertex AI · Gemini	SageMaker · Bedrock

Learn the capability first; translate product names second.

How cloud data capabilities fit together

Ingest / events

GCP: Pub/Sub · Dataflow

AWS: Kinesis · Firehose

Store

GCP: Cloud Storage

AWS: Amazon S3

Process / transform

GCP: Dataflow · Dataproc

AWS: Glue · EMR

Query / serve

GCP: BigQuery

AWS: Athena · Redshift

Orchestrate

GCP: Cloud Composer

AWS: MWAA · Step Functions

Govern / secure

GCP: IAM · Knowledge Catalog

AWS: IAM · Lake Formation

OLTP vs OLAP: two different jobs

OLTP: run the business

Operational systems handle many small reads/writes: create a policy, update a claim, record a payment. They optimize for transactions, consistency, and low-latency application behavior.

Typical stores: **Cloud SQL, AlloyDB, Spanner, RDS, Aurora.**

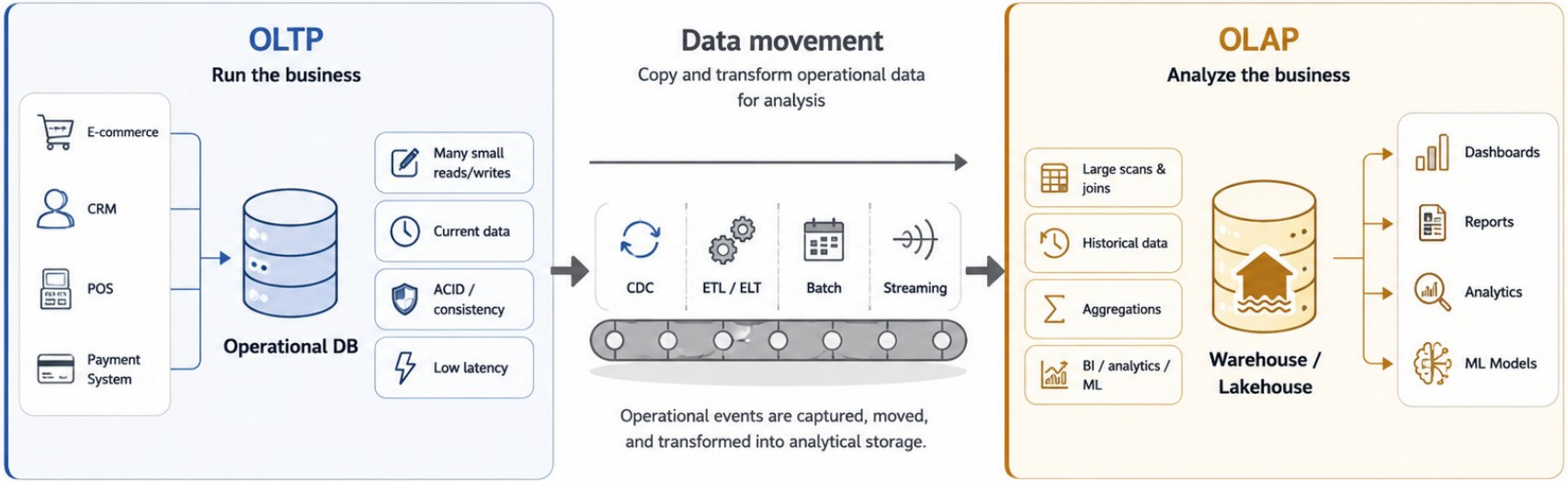
OLAP: analyze the business

Analytical systems scan, join, aggregate, and summarize large historical datasets. They optimize for columnar reads, BI, reporting, ML features, and cost-aware query execution.

Typical stores: **BigQuery, Snowflake, Redshift, Athena**.

Foundation rule: operational stores capture events; analytical stores turn many events into decisions. Do not design one system to be great at both unless the requirements justify it.

OLTP vs OLAP: how they work together



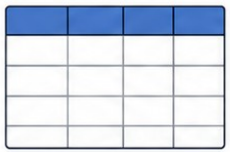
Example:
Customer places an order → transaction stored in OLTP → data replicated and modeled → dashboards show sales trends in OLAP.

Best practice: OLTP runs the business. OLAP analyzes the business. They usually work together, not as one system.

Purpose-built databases exist

Common data models and what they are best known for

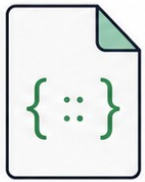
1



Relational

Tables, rows, SQL, transactions

2



Document

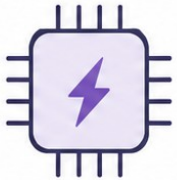
Flexible JSON-like documents

3

Key-Value

Simple key to value lookups

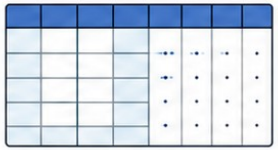
4



Cache / Memory

Ultra-fast in-memory access

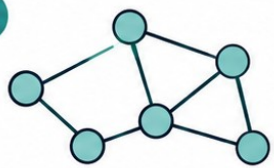
5



Wide Column

Massive scale, sparse columns

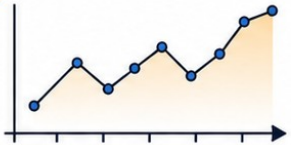
6



Graph

Nodes, edges, relationships

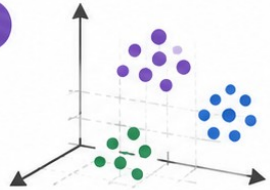
7



Time Series

Events and metrics over time

8



Vector

Embeddings and similarity search

Why purpose-built databases exist

- Relational databases are still central, but not every workload is a neat table + transaction.
- New data shapes and access patterns created specialized stores: JSON documents, key lookups, sparse columns, relationships, telemetry, embeddings, and cache state.
- The design move is not “SQL or NoSQL?” The move is: what access pattern must be fast, reliable, and affordable?

Flexible records

Profiles, device payloads, changing attributes
Document Stores

Fast lookup

Session, cart, feature flag, token state
Key-Value / Cache

Huge sparse scale

Telemetry, entity history, high write volume
Wide-Column

Relationships

Fraud rings, lineage, recommendations
Graph

Time changes

Metrics, IoT, observability, driving events
Time-Series

Similarity

Search by meaning, RAG, deduping text
Vector/Search

Database families: choose the model first

Family	Optimized for	Example use case
Relational OLTP	Transactions, constraints, strong schema	Policy admin, billing, claims app records
Warehouse / OLAP	Large scans, joins, aggregates, BI	Monthly claims analytics, taxi demand metrics
Document	Flexible JSON-like records	Customer profile with changing preferences
Key-value / cache	Lookup by key at very low latency	Session state, feature flags, API response cache
Wide-column	Massive sparse rows and high write scale	Vehicle telemetry by device and timestamp
Graph	Relationship traversal and pattern matching	Fraud rings, data lineage, customer household graph
Time-series	Time-window queries over changing signals	IoT metrics, driving events, observability
Vector / search	Similarity, semantic search, retrieval	Find similar claim notes or retrieve RAG context

DATABASE SERVICES

Operational stores: relational, document, key-value, wide-column

Relational OLTP

Use when correctness, relationships, transactions, and SQL constraints matter.

GCP: Cloud SQL · AlloyDB · Spanner
AWS: RDS · Aurora

Document

Use when records are nested, flexible, and application-shaped rather than table-shaped.

GCP: Firestore
AWS: DocumentDB

Key-value / cache

Use when the access pattern is “given this key, return this value now.”

GCP: Memorystore · Bigtable patterns
AWS: DynamoDB · ElastiCache

Wide-column

Use for huge sparse datasets, high write throughput, and predictable row-key access.

GCP: Bigtable
AWS: Keyspaces

Analytical and specialized stores

Warehouse / OLAP

Curated analytical SQL, BI, reporting, aggregate queries, cost-aware scans.

GCP: BigQuery · Snowflake
AWS: Redshift · Athena

Graph

Relationship-first questions: fraud, network traversal, lineage, recommendations.

GCP: Spanner Graph
AWS: Neptune

Time-series

Signals over time: telemetry, monitoring, IoT, sensor history.

GCP: BigQuery/Bigtable patterns
AWS: Timestream

Vector / search

Similarity and semantic retrieval over text, images, and embeddings.

GCP: AlloyDB AI · BigQuery vector search · Firestore vector
AWS: Aurora pgvector · OpenSearch

DATABASE SERVICES

		Google Cloud	AWS
1	Operational relational	Cloud SQL • AlloyDB • Spanner	RDS • Aurora
2	Analytical warehouse	BigQuery • Snowflake	Redshift • Athena
3	Document data	Firestore	DocumentDB
4	Key-value / cache	Memorystore • Bigtable patterns	DynamoDB • ElastiCache
5	Wide-column	Bigtable	Keyspaces
6	Graph relationships	Spanner Graph	Neptune
7	Time-series	BigQuery / Bigtable patterns	Timestream
8	Vector / semantic search	AlloyDB AI • BigQuery vector • Firestore vector	Aurora pgvector • OpenSearch

Use this as a mental map for common service choices in data engineering.

CLOUD VALUE

One platform, many managed data choices

What cloud changes

You do not start by buying servers, installing database software, designing backup jobs, and negotiating every vendor from scratch. You start from managed building blocks that already integrate with IAM, networking, billing, monitoring, and automation.

Faster experimentation

Try the right database family quickly; prove value before hardening.

Common governance

IAM, audit logs, regions, encryption, billing, and quotas are consistent patterns.

Less undifferentiated work

Backups, patching, scaling, and availability become provider-managed responsibilities.

But the engineering judgment does not disappear: you still choose the service based on access pattern, data sensitivity, latency, cost, and operational risk.

The compute spectrum: where your code runs

Aspect	VM	Container	Serverless	Managed
You manage	**OS + runtime**	**Image + scale**	**Just your code**	Config + data
Scaling	**Manual**	**Set min / max**	Auto, to zero	Automatic
Best DE fit	**Legacy deps**	Custom long jobs	Event glue, ETL	Pipelines & BI
GCP / AWS	**GCE / EC2**	Cloud Run / Fargate	Functions / Lambda	Dataflow / Glue

Rule of thumb: pick the most managed option that fits. Less to manage means less to break, patch, and pay for while idle.

The compute spectrum: where your code runs

More you manage ⟷ Less you manage

VM

You manage everything below the app
(OS, runtime, patches, networking) - essentially IaaS

Container

The cloud handles the hardware and OS; you manage the image and scaling rules
datacenterknowledge

Serverless

The cloud handles everything except your code; runtime is virtualized away entirely
web.cecs.pdx.edu

Managed

The cloud handles the compute engine itself; you only configure the pipeline and supply data

Rule of thumb: pick the most managed option that fits. Less to manage means less to break, patch, and pay for while idle.

Where your pipeline actually runs

Scheduled batch ETL

GCP: Cloud Run jobs · Batch

AWS: Lambda · AWS Batch

Streaming pipeline

GCP: Dataflow

AWS: Kinesis · Glue

Big Spark jobs

GCP: Dataproc · Serverless

AWS: EMR

Notebooks / ad hoc

GCP: Vertex AI Workbench

AWS: SageMaker Studio

Always-on service

GCP: Cloud Run · GKE

AWS: Fargate · EKS

Event glue / small tasks

GCP: Cloud Functions

AWS: Lambda

Networking and the egress trap

Your private network (VPC)

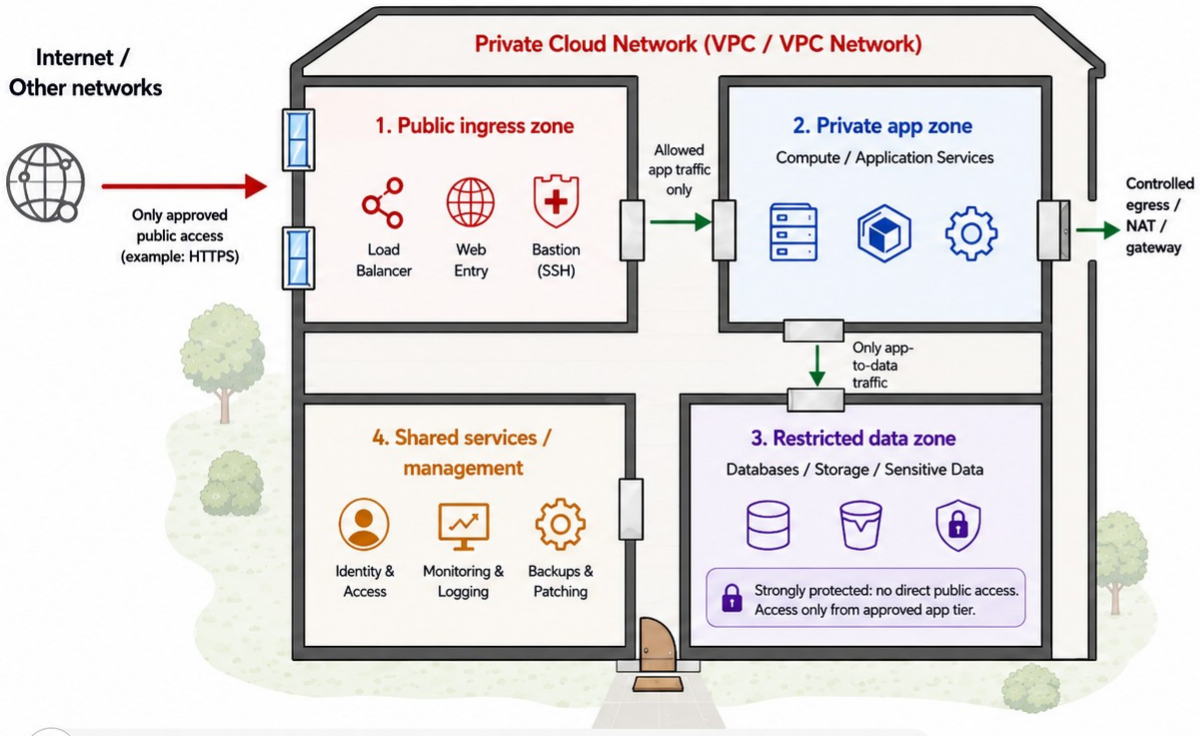
Private by default	Public (opt in)	Egress costs

Data coming in is usually free. Data going out, or across regions, is billed per GB.

- Resources start private. You open public access on purpose, never by accident.
- Egress, meaning data leaving a region or the cloud, is the cost beginners miss most.
- Co-locate compute and storage in one region to keep pipelines fast and cheap.

Networking and Network Security: a house floor plan analogy

Cloud-agnostic concepts for AWS and GCP



Legend: House Analogy → Networking Concept

House boundary	=	Private network isolation
Rooms	=	Subnets / security zones
Doors & locks	=	Firewall rules / security groups
Windows	=	Public endpoints (Only in public ingress zone)
Hallways	=	Allowed east-west traffic
Side exit	=	Controlled egress

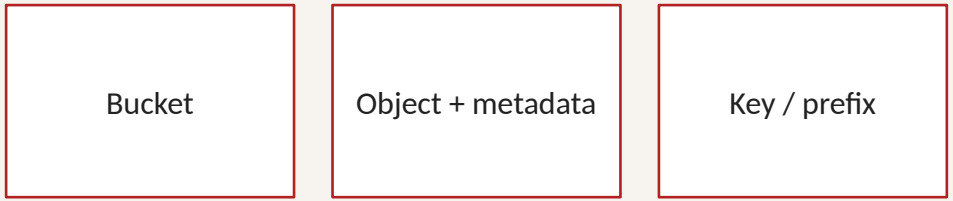
Why it matters

- Public access is intentional and limited to the public ingress zone.
- Private zones are segmented and not directly exposed.
- Limit who can talk to what.
- Protect sensitive data in deeper private zones.
- Reduce risk, improve governance, and control egress cost.

Design for least privilege: only open what's necessary, monitor continuously, and review rules regularly.

Object storage is the cloud landing zone

Three primitives



Store any format at elastic scale: CSV, JSON, Parquet, images, and documents.

- Object storage (GCS, S3) is where raw data lands first, in any format, at huge scale. It is the foundation of a data lake and the start of almost every pipeline.
- Full deep dive on Day 4 (Data at Rest): formats, file layout, lifecycle, and querying data in place.

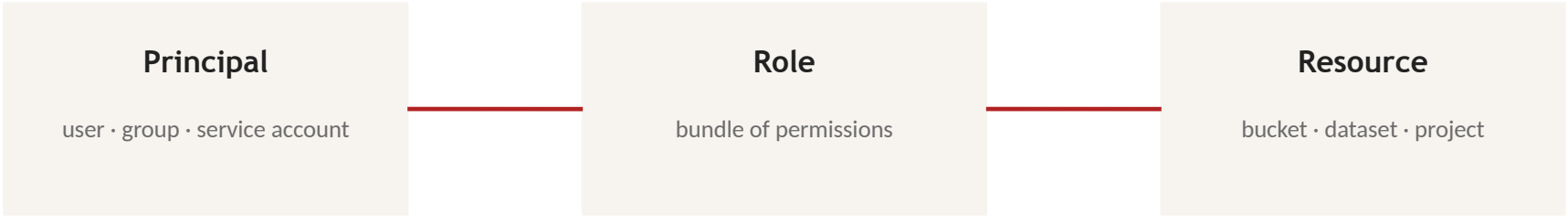
02

Identity & access management

Least privilege is a habit, not a feature.

IAM in one picture

“WHO can do WHAT on WHICH resource”



- **Service accounts: identities for programs, not people. Your pipelines will run as one.**
- **Roles in GCP: basic (avoid in production), predefined (use these), custom (when needed).**
- **Least privilege: grant the smallest role that gets the job done, at the narrowest scope.**

How your code signs in

User identity (a person)

Signs in with email and MFA

Used in the console and CLI

Great for development

Not for automated pipelines

Service account (a program)

An identity for code, not a person

Your pipeline runs as one

Gets its own IAM roles

No password to type

ADC (Application Default Credentials): your code finds the right identity automatically, from the environment, an attached service account, or your gcloud login.

Keys vs keyless: prefer keyless

Service account keys (avoid)

A downloadable JSON secret

Works anywhere, which is the risk

Leaks end up in Git and logs

Hard to rotate and audit

Keyless / workload identity (prefer)

No secret to store or leak

Identity attached to the workload

Short-lived, auto-rotated tokens

The modern default on GCP and AWS

Least privilege: give a pipeline only the roles it needs, at the narrowest scope. A leaked key with broad access is how breaches happen.

03

Billing and cost control

Spend on purpose, not by accident.

COST CONTROL

Billing without surprises

Budgets & alerts

Set a budget per project; alert at 50/90/100%, the first thing in any new project

Labels & projects

Tag resources by team/purpose so costs are attributable

Free tiers

BigQuery: 1 TB queries + 10 GB storage monthly; GCS 5 GB; always-free VM, enough for our labs

Quotas

Hard caps on resource use: protective rails, not obstacles

COST CONTROL

Five ways data engineers burn money

Mistake	Why it hurts	Habit that prevents it
SELECT * on huge tables	BigQuery bills by bytes scanned	Select only needed columns; preview first
Idle clusters	Spark clusters bill per minute, used or not	Auto-terminate; ephemeral clusters
Cross-region egress	Moving data out of region costs per GB	Co-locate storage and compute
Forgotten resources	Orphaned disks, old buckets accumulate	Clean up after every lab
No partitioning	Full scans on date-filtered queries	Partition by date (Week 3)

DEMO

Instructor orientation: GCP console

- **Console anatomy: project picker, navigation, search, Cloud Shell**
  - Find IAM: principals, roles, and service accounts
  - Find Billing: budgets, reports, and quotas
  - Find BigQuery Studio: Explorer, editor, and job information
  - Do not run the learner query yet; Activity 1 starts from a clean Sandbox project

ACTIVITY 1

Activity 1: BigQuery Sandbox

Goal: experience a managed warehouse and connect query choices to bytes processed.

60 min

1. Enable Sandbox with no billing account.
2. Inspect the NYC Citi Bike public table.
3. Run the provided station query and map the result.
4. Compare one-column and SELECT * estimates; predict before changing LIMIT.

Deliverable: Q1–Q7, validator estimates, and the exit ticket in your lab notes.

Activity 2: You're the architect

Goal: map business requirements to capabilities and defend GCP and AWS service choices.

75 min

1. Receive one primary scenario and assign team roles.
2. Decide batch, streaming, or separate paths; connect services in execution order.
3. Identify the trigger, provider responsibilities, cost risk, and PII control.
4. Reject one alternative and prepare a five-minute readout; everyone speaks.

Deliverable: completed worksheet, team readout, and one peer challenge.

Activity 3: Estimate the bill

Goal: estimate a workload's monthly cost and design the budget you would set for it.

45 min

1. Open the Google Cloud Pricing Calculator.
2. Price one Activity 2 scenario: storage, queries, and compute.
3. Decide the budget amount and 50/90/100% alert thresholds.
4. Name the biggest cost driver and one way to cut it.

Deliverable: a cost estimate, your budget plan, and the top cost driver in your notes.

RECAP

Key takeaways

1. Learn capabilities, then translate product names across clouds.
2. Projects and accounts are your billing and IAM boundary, one per team or environment.
3. Match the workload to the compute: VM, container, serverless function, or managed service.
4. Pipelines authenticate as service accounts. Prefer keyless identity and least privilege.
5. Watch egress and idle compute, and set a budget before you build. Storage deep dive is Day 4.